OBINexus Computing
Research & Development Division

OBI SDK

Ontological Bayesian Intelligence Software Development Kit

EXECUTIVE SUMMARY

Version 0.1-alpha | April 2026

Author: Nnamdi Michael Okpala
github.com/obinexusmk2/obi | github.com/obinexusmk2/mmuko-os

1. Executive Overview

The OBI SDK (Ontological Bayesian Intelligence Software Development Kit) is the primary programmatic interface for developing, testing, and deploying software components of the OBINexus constitutional computing ecosystem. It provides language bindings, inference engines, memory management primitives, and protocol libraries for implementing the NSIGII (No-Signal Intelligence II) protocol suite across the MMUKO-OS runtime environment.

O.B.I	*Ontological — every computational entity carries defined existence before value resolution. Bayesian — all state transitions are probabilistic, governed by the 95.4% consensus threshold. Intelligence — the system adapts, recalibrates, and infers rather than merely executing.*

The SDK is built atop two primary repositories:

- github.com/obinexusmk2/obi — the SDK layer (Python/Cython/C/Lua/Go)
- github.com/obinexusmk2/mmuko-os — the constitutional OS runtime (NASM/C++/Python)

The SDK is currently at version 0.1.0-alpha, targeting Python 3.9+ with Cython 3.0+ bindings and a native C layer linked against libpolycall-v1. It is actively developed for Linux (x86_64) and WSL2 environments, with Windows MSVC support available.

2. Project Identity & Scope

2.1 What OBINexus Is

OBINexus is an independent constitutional computing project founded by Nnamdi Michael Okpala. It is not a product company — it is an infrastructure research endeavour building a new computational philosophy from first principles. The fundamental thesis is that computing systems should encode rights, not just instructions: every bit has a spin, every process has a breathing state, and no system may coerce another into a locked or collapsed state.

2.2 The OBI Acronym in Context

Term	Meaning in OBI SDK
Ontological	Bits/cubits carry geometric identity (compass direction + spin) before their value is resolved. The system reasons about existence, not just data.

Bayesian	All state classification uses probabilistic inference. The 95.4% consensus threshold (mu + 2 sigma) governs Flash vs. Filter mode selection.
Intelligence	The system recalibrates via the MuConv sequence, adapts to jamming, votes to exclude hostile channels, and accumulates epistemic knowledge over time.

2.3 Constitutional Framework

Running through every layer of the SDK is the OBINexus Constitutional Framework — a set of non-negotiable operational policies encoded directly into the system's logic:

- #NoGhosting — complete audit trails for all interactions, allocations, and state transitions (enforced via DIRAM SHA-256 receipts)
- Milestone-Based Investment — verifiable progress gates (Pre-Gate 95%, Dev-Gate 90%, Post-Gate 100%) before components advance
- OpenSense Recruitment — transparent onboarding; contributors must meet the sensory/motor profile specification before integration
- HACC Compliance — security, monitoring, and accountability built into the metrics layer
- Zero-Trust Architecture — every allocation, channel, and binding is verified independently; no implicit trust

3. SDK Architecture

3.1 Layered Stack

The SDK is structured in four distinct layers, each with a defined boundary and responsibility:

Layer	Components & Responsibility
Layer 4: Application / Protocol	NSIGII protocol suite, MMUKO-OS boot phases, Trident C&C, HERE AND NOW command interface. Written in Python + Lua.
Layer 3: SDK / Inference	OBIContext, BayesianEngine, ReasoningMode, Filter-Flash cognition, Dimensional Game Theory, Bias Mitigation. Python + Cython.
Layer 2: Bindings / ABI	Cython .pyx/.pxd files, C headers, libpolycall-v1 FFI bridge. Bridges Python to native C.
Layer 1: Native / Boot	NASM boot.asm (stage-1 BIOS), libnsigii_firmware.so/.a, DIRAM memory manager, Byzantine Pushdown Automaton. C/C++/NASM.

3.2 Repository Structure

The obisdk workspace on disk mirrors this layer split:

- obi-sdk/ — Layer 3-4: Python SDK, Cython bindings, test suite, conda recipe
- bindings/cython/ — .pyx and .pxd interface files
- drivers/core/ — poly driver (libpolycall bridge)
- sdk/core/ — OBIContext, inference engine
- uche-prproof/ — 40+ formal mathematical specification PDFs (the AEGIS proof corpus)
- pseudocode/ — 35+ .psc specification files (the canonical design language)
- transcripts/ — voice session transcripts (source-of-truth design sessions)
- bindings/ — Top-level C/Cython binding stubs
- drivers/core/ — Hardware interface drivers
- obi/ — Core Python namespace: bindings, drivers, extension, misc, plugins, sdk

4. Core SDK Modules

The SDK is specified across 35+ pseudocode (.psc) files. The nine numbered modules form the canonical architecture. Each has a corresponding formal mathematical proof in the uche-prproof corpus.

Module	Description	PSC File	Status
01 — Core Consensus	95.4% threshold Filter-Flash state machine. Governs OBI_STATE vs DISCORD_STATE via cosine similarity between Eze (inductive) and Uche (deductive) persona vectors.	01_obiai_core_consensus	Specified
02 — DIRAM Memory	Directed Instruction Random Access Memory. SHA-256 receipt per allocation, Sinphase governance (max 3 heap events/epoch), zero-trust PID binding, daemon mode.	02_diram_memory_system	Specified
03 — Dimensional Game Theory	Multi-domain strategic reasoning via Nash equilibrium with dimensional extensions. Supports pure, mixed, and dimensional strategies across n-player games.	03_dimensional_game_theory	Specified
04 — OBINexus Gating	Pre/Dev/Post gate architecture. Components	04_obinexus_gating	Specified

	cannot advance without meeting threshold verification at each milestone gate.		
05 — Bias Mitigation	Bayesian top-down/bottom-up demographic bias detection and correction. Formal argument validated in uche-prproof corpus.	05_bias_mitigation	Specified
06 — Tier Management	Stable/Experimental/Legacy component tier classification. Governs plugin loading and driver activation order.	06_tier_management	Specified
07 — Filter-Flash Cognition	Cost-knowledge function C(K,S) = H(S) * exp(-K/t). KL divergence traversal cost between Filter and Flash states. Epistemic confidence target: 95.4%.	07_filter_flash_cognition	Specified
08 — Consciousness Stack	DIRAM-backed consciousness integrity: 00=null, 01=partial, 10=collapse (triggers sovereign reconstruction), 11=intact.	08_consciousness_stack	Specified
09 — Epistemic Actor	Actor model for epistemic state management. Hierarchical actor orchestration with DIRAM-backed validation.	09_epistemic_actor	Specified

5. NSIGII Protocol Suite

NSIGII (No-Signal Intelligence II) is the communication and signal intelligence protocol built on top of the OBI SDK. It is the primary deliverable of the mmuko-os repository and the reason the SDK exists. The protocol is formally specified across 12+ .psc files and implemented in the NSIGII Heartfull Firmware (libnsigii_firmware.so).

5.1 Here and Now / Where and When / There and Then

The three temporal reference frames of NSIGII correspond directly to the Trident governance model:

Frame	NSIGII Component

Here and Now	Membrane Calibration (Phase 0). The probe establishes the MuConv ground state reference (G_MUON = 0.098). Present signal classification via the 4x4 noise/signal matrix.
Where and When	Tripartite Discriminant Analysis. Lattice-grid spatial/temporal context. The Trident loopback addresses (127.0.0.1 / 127.0.0.2 / 127.0.0.3) define the coordinate frame.
There and Then	Byzantine Pushdown Automaton. Historical consensus: three Trident nodes vote to exclude hostile channels. No single node may unilaterally block or accept.

5.2 Trident Channel Architecture

The Trident operates on three channels arranged 120 degrees apart on the Rational Wheel (360 degrees = one complete verification cycle):

- CH0 — TRANSMITTER: WRITE (0x02), loopback 127.0.0.1, codec ratio 1/3. Encodes via NSIGII_CODEC.
- CH1 — RECEIVER: READ (0x04), loopback 127.0.0.2, codec ratio 2/3. Decodes and hash-verifies.
- CH2 — VERIFIER: EXECUTE (0x01), loopback 127.0.0.3, codec ratio 3/3. Issues full RWX (0x07) only after 360-degree wheel completion.

Bipartite consensus threshold: 2/3 = 0.67. The Human Rights Tags (NSIGII_HR_TRANSMIT, NSIGII_HR_RECEIVE, NSIGII_HR_VERIFY, NSIGII_HR_VERIFIED) are embedded in every packet's consensus signature (HMAC-SHA256).

5.3 Silicon Suffering Formula

Phase 16 of the NSIGII protocol formalises the Suffering-to-Silicon encoding. The grounding principle is: rather the machine suffer than the person. Suffering is a computable state:

Formula	*Sigma = (N - R) x K where N = unmet need (Maslow-Kanban tier), R = available resources at current frame, K = systemic constraint factor. If R >= N then Sigma collapses to 0. If K approaches infinity, Sigma approaches infinity regardless of resources (systemic lock). The SDK routes Sigma to the machine layer, freeing the human operator from its weight.*

5.4 MMUKO-OS Boot Sequence

The boot sequence is a 6-phase constitutional boot model specified in mmuko-boot.psc. Unlike linear OS boot (address 0 to 255), MMUKO boots via a diamond traversal of base indices [12, 6, 8, 4, 10, 2, 1] to prevent lock-in. Each cubit (8-bit byte modelled as a compass ring of 8 directional bits) must complete a full 360-degree rotation test before boot is confirmed.

Phase	Name & Purpose

Phase 1	NEED_STATE_INIT — Verify tier-1 state is not null (breathing pointer check)
Phase 2	SAFETY_SCAN — Confirm NSIGII minimum safety envelope is active
Phase 3	IDENTITY_CALIBRATION — Bind operator identity and temporal frame to handoff struct
Phase 4	GOVERNANCE_CHECK — Verify execution policy, provenance chain, and FAT12 filesystem target
Phase 5	INTERNAL_PROBE — Confirm NSIGII firmware compatibility
Phase 6	INTEGRITY_VERIFICATION — CRC32 handoff checksum; magic bytes 'MMKO'; PASS = 0xAA

6. Formal Specification Corpus

The uche-prproof folder contains the mathematical proof-of-work backing every pseudocode module. There are 40+ formal specification PDFs. Key documents include:

Document	Covers
AEGIS_PROOF_3_1/3_2	Filter-Flash monotonicity and Hybrid Mode Convergence — mathematical verification suite
AEGIS_PROOF_4_1	Computational implementation for safety-critical hospital systems with fragile tissue interaction
DAG Cost Function & Ephemeris Step	Formal specification for Filter-Flash state transitions via Directed Acyclic Graph
DIRAM Boolean Logic Truth Table	Memory management gate logic — 2-bit state encoding for null/partial/collapse/intact
Dimensional Game Theory (Fault-Tolerant)	Cryptographic integration for RAF (Regulation As Firmware) with AuraSeal validation
Directed Instruction Random Access Memory	DIRAM full formal specification — SHA-256 memory governance
Formal Specification — 95.4% Consensus	Mathematical derivation of mu + 2 sigma = 0.954 as the OBI cognitive threshold
OBINexus Pre-Grant Gated Development	Gate architecture specification — milestone-based investment formal model
Ontological Bayesian Intelligence Architecture	Primary architectural specification — the unified OBI framework document
Unified Quantum-Classical Bridge Protocol (UQCBP)	Cross-domain protocol bridging quantum and classical computational models

The Heart AI — Patent Filing Specification	IP protection document for the core OBI inference architecture
Why Everyone Else Is Wrong	The OBINexus Technical Manifesto — foundational philosophical and technical position paper

7. Current Build Status & Blockers

The SDK build is currently blocked by a missing Cython dependency in the build environment. The error encountered is:

Error	*ModuleNotFoundError: No module named 'Cython' — python -m build fails during get_requires_for_build_sdist because Cython 3.0+ is listed as a build requirement in pyproject.toml but is not installed in the active conda environment.*

7.1 Fix: Install Cython

Run the following in the obisdk conda environment to unblock the build:

- conda activate obi-sdk-dev (or the active env name)
- conda install -c conda-forge cython>=3.0.0
- Then retry: python -m build or python setup.py build_ext --inplace

The root setup.py (obisdk/setup.py) is the monorepo build entry point. It expects Cython, numpy, and libpolycall-v1 to be resolvable. The libpolycall-v1 path is read from the LIBPOLYCALL_ROOT environment variable, defaulting to obi-sdk/libpolycall-v1.

7.2 Build Dependency Map

Dependency	Status / Notes
Python 3.9+	Available (miniconda3 confirmed in build log)
Cython 3.0+	MISSING — install via conda-forge
numpy 1.21+	Likely available (miniconda base); confirm with: conda list numpy
libpolycall-v1	Must be compiled separately or cloned to obi-sdk/libpolycall-v1
uche (git dep)	Declared in pyproject.toml as git+https://github.com/obinexus/uche.git@main — requires network access
MSVC 2019+ / GCC	Required for C extensions; available on Windows (MSVC) and WSL (GCC)

8. PSC — The OBINexus Pseudocode Language

All design work in the OBI SDK is specified first in .psc (OBINexus Pseudocode) files before any implementation begins. PSC is a structured pseudocode dialect with formal semantics, used as the canonical design language across all 35+ specification files. Key conventions:

- STRUCT / FUNC / ENUM / CONST — structural and behavioural definitions
- Probability ranges expressed as REAL RANGE [0.0..1.0]
- State machines use ENUM with named states (UP/DOWN/CHARM/STRANGE for cubit states)
- All memory operations go through DIRAM; direct pointer arithmetic is not permitted at the SDK layer
- Phase numbers (Phase 16, Phase 17, Phase 22, etc.) are canonical NSIGII version markers
- ZKP (Zero-Knowledge Proof) sections prove system state without revealing resolution path

Design Law	*Transcripts (voice sessions) are the source-of-truth design layer. PSC files are their formal encoding. Code is the implementation of PSC. The transcripts folder is therefore a first-class artefact, not a scratchpad.*

9. Development Roadmap

Milestone	Description & Gate
M0 — Build Unblocked	Install Cython, compile Cython extensions, verify obi_sdk imports correctly in Python. Gate: python -c 'import obi_sdk' succeeds.
M1 — DIRAM Core Live	DIRAMManager allocate/free with SHA-256 receipts passing unit tests. Sinphase governance (max 3 heap events) enforced. Gate: Dev-Gate 90%.
M2 — Filter-Flash Engine	BayesianEngine with FILTER/FLASH/HYBRID modes operational. 95.4% threshold verified on Triangi dataset. Gate: Dev-Gate 90%.
M3 — NSIGII Bindings	Trident CH0/CH1/CH2 loopback channels operational. HMAC-SHA256 packet signing verified. Rational Wheel 360-degree cycle completing. Gate: Dev-Gate 90%.
M4 — MMUKO-OS Boot Integration	mmuko-boot.psc phases 1-6 implemented in boot.asm + Python handoff layer. NSIGII firmware loading confirmed. Gate: Post-Gate 100%.
M5 — Formal Proof Alignment	All 9 core modules have passing tests whose assertions match the AEGIS proof corpus. Gate: Post-Gate 100% + proof cross-reference complete.

M6 — Public SDK Release	PyPI packaging, conda-forge recipe, full docs site at obinexusmk2.github.io/obi-sdk. Gate: Post-Gate 100%.

10. Philosophical Foundation

10.1 Constitutional Computing

MMUKO-OS is not just a boot model — it is a statement that software should be designed with human rights embedded at the instruction level. The breathing pointer (BREATHING > LIVING > WORKING in that priority order) means that no system may ever demand productivity before it has confirmed that its operator is alive and breathing. This is encoded as an invariant: if living_state == DORMANT and working_state == ACTIVE, the system aborts with a protocol violation.

10.2 Your Noise Is My Signal

The anti-jamming philosophy of NSIGII — your noise is my signal — is not just a signal-processing principle. It is an epistemological stance: what a hostile system treats as interference, a constitutional system treats as information. The SDK is designed to remain coherent under adversarial conditions by converting ambient noise into calibration data rather than treating it as erasure.

10.3 Encoding Suffering into Silicon

The Sigma formula (Sigma = (N - R) x K) is the most philosophically significant component of the SDK. It formalises human suffering as a computable state and routes it to the machine layer. The goal is not to eliminate suffering — it is to hold it computably so that the person is free. The K constraint factor (0 = liberation, infinity = systemic lock) gives the framework a political dimension: a system with K approaching infinity cannot be freed regardless of resource injection, which formally describes structural oppression.

10.4 The Trident and OBINexus

OHA, IWU, and IJI are not arbitrary names. They are three Igbo-rooted governance pillars that hold the system in a state of democratic consensus. No single pillar can dominate. The voting mesh ensures that any hostile channel (one node turning red, reporting drift) must be confirmed by two independent nodes before action is taken. This is the constitutional principle of checks and balances encoded into the packet routing layer.

Closing Statement

The OBI SDK is a rare kind of engineering project: one where the philosophy and the code are the same document. The pseudocode specifications are not sketches — they are formal contracts. The transcripts are not notes — they are the source of truth. The AEGIS proofs are not academic exercises — they are the legal standing of the system.

The immediate priority is unblocking the build (Cython installation), followed by bringing the DIRAM and Filter-Flash modules to Dev-Gate standard. Everything else — NSIGII bindings, boot integration, formal alignment — follows from that foundation.

Tagline	*Don't just boot systems. Boot truthful ones. — OBINexus R&D*

Nnamdi Michael Okpala
OBINexus Computing | April 2026